



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

RISC-V Instruction Formats II

Kris Kross - Jump



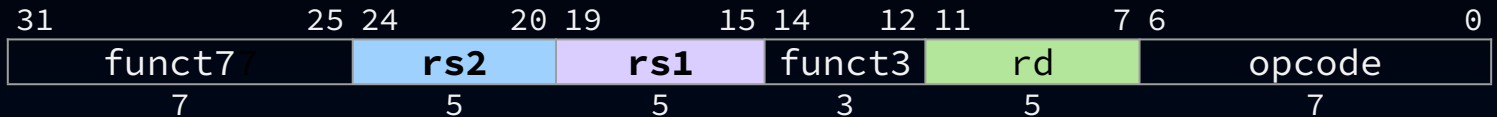
From Last Time

- What did we do last Friday? What did you learn about R-Type, I-Type, S-Type?

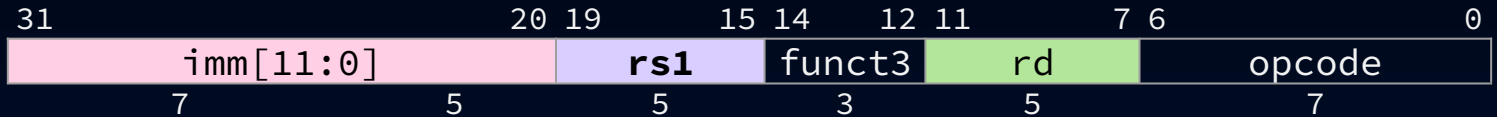


From Last Time

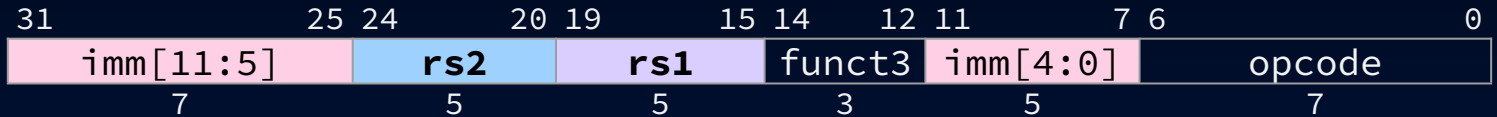
R-Type



I-Type



S-Type



Why split up the immediate field in S-Type?

- RISC-V ISA design prioritizes keeping **register fields** in the same places:
 - "Read" registers in same bitrange across format types
 - "Write" registers, too
 - Immediates less critical

Critical for simple hardware design! (more later)

Yan, SP26



Addressing Modes

- Updating the Program Counter (PC)
- B-Type Instructions
- Jump Instructions
- Jump Instruction II: `jalr`
- U-Type Instructions:
`lui`, `auipc`



Program Counter in action

[review]

pc ~~0x0000 0008~~
0x0000 000c



```
0x0  ...
0x4  .. # li x10 0x34FF
0x8  slli x12 x10 0x10
0xc  srli x12 x12 0x08
0x10 and x12 x12 x10
```

Program execution:

- Fetch instruction from memory using PC
- Execute instruction
- **Update PC** to next instruction

By default, the PC is updated to PC + 4 bytes ("next sequential instruction") **unless otherwise stated by the instruction itself.**





PC-Relative Addressing

- In almost all cases, instructions update the PC with **PC-Relative Addressing** by supplying a signed offset. $PC = PC + \text{offset}$
 - Why? **Position-Independent Code**. If all of code moves, these relative offsets won't change!
 - Arithmetic/load/store:
 $\text{offset} = +4$
 - Branches, most jumps

Labels don't actually "exist"
in machine code!

Jumps are (often) offsets relative to the PC



What is the PC-relative offset (in bytes)?

$$PC = PC + \text{offset}$$

```
0x0c Loop: beq x19 x10 End
0x10      add x18 x18 x10
0x14      addi x19 x19 -1
0x18      j    Loop
0x1c End:  ... # target instr
```

1. beq x19 x10 End
(assume branch is taken)

2. j Loop

- | | 1. | 2. |
|----|----------------|----|
| A. | +12-16 | |
| B. | +12-12 | |
| C. | +16-16 | |
| D. | +16-12 | |
| E. | Something else | |



Jumps are (often) offsets relative to the PC



What is the PC-relative offset (in bytes)?

$$PC = PC + \text{offset}$$

0x0c	Loop:	beq	x19	x10	End	}
0x10		add	x18	x18	x10	
0x14		addi	x19	x19	-1	
0x18		j	Loop			
0x1c	End:	... # target instr				

1. 2.
D. +16-12

1. beq x19 x10 End
(assume branch is taken)

pc ~~0x0000 000c~~
0x0000 001c

2. j Loop

pc ~~0x0000 0018~~
0x0000 000c



PC-Relative Addressing

- In almost all cases, instructions update the PC with **PC-Relative Addressing** by supplying a signed offset. $PC = PC + \text{offset}$
- Contrast with: **Absolute Addressing**. $PC = R[rs1] + \text{imm}$
 - Supply new address value to overwrite PC.
 - Use sparingly (only **jalr**):
 - need to build 32-bit immediate
 - position-dependent: brittle to code movement (more later)



B-Type Instructions

- Updating the Program Counter (PC)
- B-Type Instructions
- Jump Instructions
- Jump Instruction II: `jalr`
- U-Type Instructions:
`lui`, `auipc`

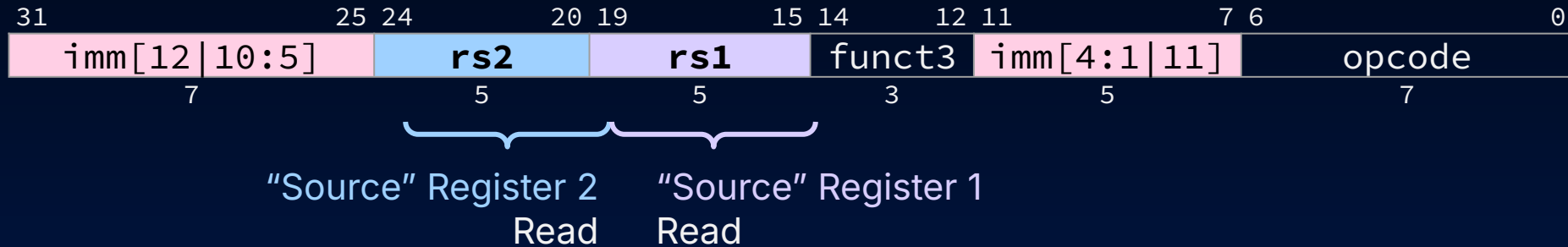


B-Type Instruction Format (1/2)

```
beq rs1 rs2 Label
  if (R[rs1] == R[rs2])
    PC = <address of Label>
      = PC + offset
```

Label: translated to PC-relative addresses, then encoded as a signed immediate offset.

branchop rs1 rs2 Label

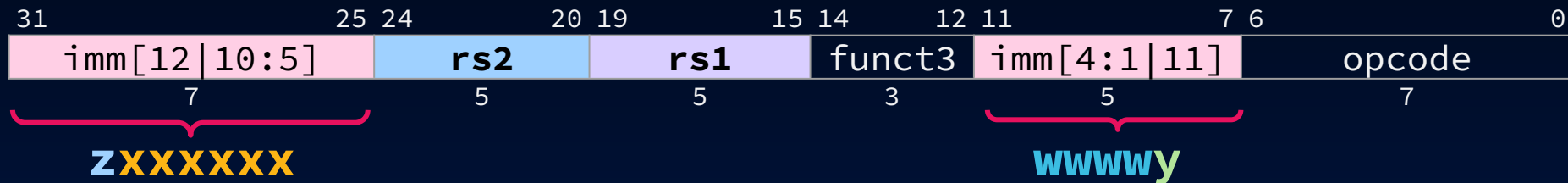




B-Type Instruction Format (2/2)

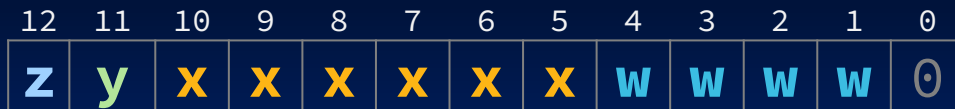
```
beq rs1 rs2 Label
  if (R[rs1] == R[rs2])
    PC = <address of Label>
      = PC + offset
```

branchop rs1 rs2 Label



offset = imm[12:0]

- imm[0] always 0
- offset is a **13-bit** integer, 2's complement



13-bit signed offset (imm)

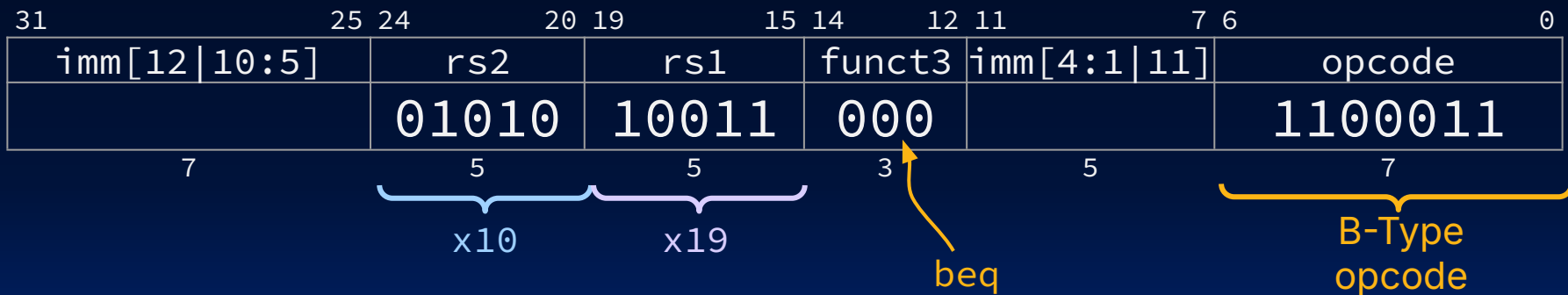
B-Type Example (1/2)

Encode the beq instruction (right):

1. Determine opcode, funct3, from reference card
2. Translate registers
3. Construct branch offset in bytes, then encode immediate.

```
Loop: beq x19 x10 End
      add x18 x18 x10
      addi x19 x19 -1
      j Loop
End:   ... # target instr
```

beq x19 x10 End





B-Type Example (2/2)

ANIMATE THIS SLIDE

Encode the beq instruction (right):

1. Determine opcode, funct3, from reference card
2. Translate registers
- ➔ 3. Construct branch offset in bytes, then encode immediate.

```
Loop: beq x19 x10 End
      add x18 x18 x10
      addi x19 x19 -1
      j Loop
End:   ... # target instr
```

beq x19 x10 End

31							25		24		20			19		15			14		12		11		7			6		0	
imm[12 10:5]							rs2					rs1					funct3			imm[4:1 11]					opcode						
00000000							01010					10011					000			10000					1100011						
7							5					5					3			5					7						

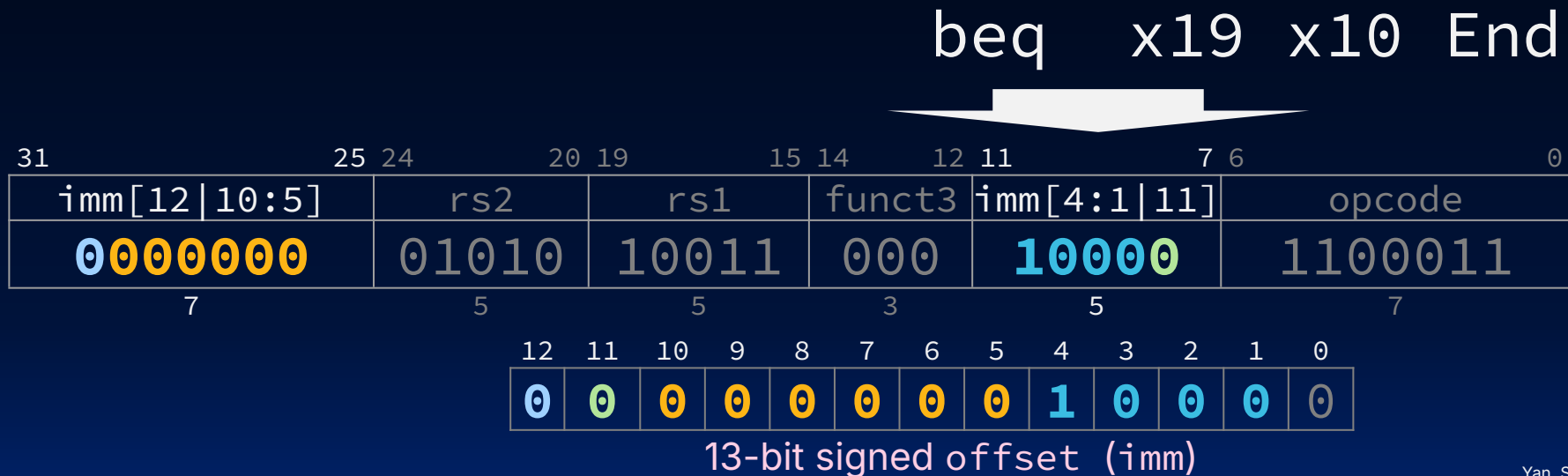
+4 instructions
= + 16 B offset

12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	1	0	0	0	0
offset												



B-Type: Design Questions

1. Why store only 12 bits of the **13-bit immediate** offset?
2. Why **"swirl"** around the immediate bits?





1. Scaling the Offset Maximizes Branch Range

RV32I: 32-bit instructions. The **13-bit signed offset** for conditional branches reaches:

- A. $\pm 2^9$ instructions from PC
- B. $\pm 2^{10}$ instructions from PC
- C. $\pm 2^{11}$ instructions from PC
- D. $\pm 2^{12}$ instructions from PC
- E. $\pm 2^{13}$ instructions from PC
- E. Something else

12	11	10	9	8	7	6	5	4	3	2	1	0
z	y	x	x	x	x	x	x	w	w	w	w	0

13-bit signed offset (imm)

offset = imm[12:0]

- imm[0] always 0
- offset is a **13-bit** integer, 2's complement
- PC = PC + offset

L13: The 13-bit signed offset for conditional branches reaches:

12	11	10	9	8	7	6	5	4	3	2	1	0
z	y	x	x	x	x	x	x	w	w	w	w	0

13-bit signed offset (imm)

offset = imm[12:0]

- imm[0] always 0
- offset is a **13-bit** integer, 2's complement
- PC = PC + offset

± 2⁹ instructions from PC.

0

± 2¹⁰ instructions from PC.

0

± 2¹¹ instructions from PC.

0

± 2¹² instructions from PC.

0

± 2¹³ instructions from PC.

0

Something else

0





1. Scaling the Offset Maximizes Branch Range: Sol

RV32I: 32-bit instructions. The **13-bit signed offset** for conditional branches reaches: **B.** $\pm 2^{10}$ instructions from PC.

12	11	10	9	8	7	6	5	4	3	2	1	0
z	y	x	x	x	x	x	x	w	w	w	w	0

13-bit signed offset (imm)

Q: But if instructions are 32-bits...why not expand jump range by encoding **word** offsets?

A: 16-bit compressed instruction extension! (see notes at 🏠)

PC relative offsets are:

- $[-2^{12}, +2^{12}-2]$ in **bytes**, spaced by 2B increments
- $[-2^{10}, +2^{10}]$ in **32-bit instructions**.



2. Swirl Immediate Bits around for HW Design

- Recall RISC-V keeps read/write register fields consistent across instruction formats.
- RISC-V also tries to keep **bit positions of immediates** consistent:

	31	26	24	20	19	15	14	12	11	7	6	0
I-Type	imm[11:0]					rs1	funct3	rd		opcode		
	imm[11:5]			imm[4:0]								
	yxxxxxx			wwwwv		10011						
S-Type	imm[11:5]			rs2		rs1	funct3	imm[4:0]		opcode		
	yxxxxxx							wwwwv				
B-Type	imm[12 10:5]			rs2		rs1	funct3	imm[4:1 11]		opcode		
	zxxxxxx							wwwwy				





B-Type Instructions: Reference

			funct3	opcode		
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	beq
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	bne
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	blt
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	bltu
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	bge
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	bgeu



Jump Instructions

- Updating the Program Counter (PC)
- B-Type Instructions
- Jump Instructions
- Jump Instruction II: `jalr`
- U-Type Instructions:
`lui`, `auipc`



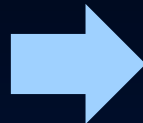
J-Type Instruction: `jal`

- **J**ump **a**nd **l**ink

`jal rd Label`

$R[rd] = PC + 4$

$PC = \text{<address of Label>}$



$R[rd] = PC + 4$

$PC = PC + \text{offset}$

- Means:

- Write $PC + 4$ to register `rd`
- Unconditional jump: go to statement labeled `Label`

Like branch instructions, `jal` uses PC-relative addressing—but with longer jumps.

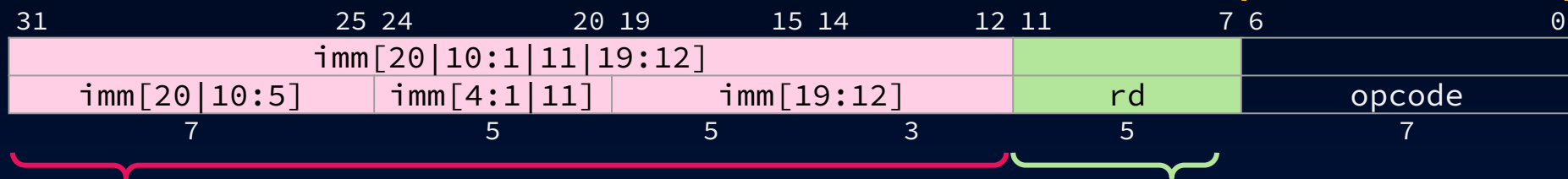


J-Type Instruction Format

- J-Type (textbook: UJ-Type) used only for **jal**.

jal rd Label

J-Type opcode:
1101111



offset = imm[21:0]

- imm[0] always 0
- offset is a **21-bit** integer, 2's complement
- "Bit swirling" to reduce HW cost (verify at 🏠)

"Destination" Register
gets "return address."
 $R[rd] = PC + 4$



RV32I: Jumps

GOT TO HERE

See reference card on [course website](#).

Which instructions/pseudoinstructions...:

1. ...are J-Type? ...are I-Type?
2. ...are used to call functions?
3. ...are used to return from functions?
4. ...are used to break out of loops?
5. ...use PC-relative addressing?
...use PC-absolute addressing?

<https://notes.cs61c.org/content/rv-procs/rv-jumps-stack/#tab-rv-jumps>

Jump **a**nd **l**ink
`jal rd Label`

Jump
(pseudo)
`j Label`
`jal x0 Label`

Jump **a**nd **l**ink
register
`jalr rd rs1 imm`

Jump **r**egister
(pseudo)
`jr rs1`
`jalr x0 rs1 0`



Yan, SP26



RV32I: Jumps

See reference card on [course website](#).

Which instructions/pseudoinstructions...:

1. ...are J-Type? ...are I-Type?
2. ...are used to call functions?
3. ...are used to return from functions?
4. ...are used to break out of loops?
5. ...use PC-relative addressing?
...use PC-absolute addressing?

J-type: `jal`, `j`;

I-type: `jalr`, `jr`

Write "return" address: `jal`, `jalr`

Read "return" address: `jr`, `jalr`

Ignore return address (write to `x0`):
`j`, `jr` (`,` `jalr`)

PC-relative (J-type): `jal`, `j`

PC-absolute (I-type): `jalr`, `jr`

Jump **a**nd **l**ink
`jal rd Label`

Jump
(pseudo)
`j Label`
`jal x0 Label`

Jump **a**nd **l**ink
register
`jalr rd rs1 imm`

Jump **r**egister
(pseudo)
`jr rs1`
`jalr x0 rs1 0`



J-Type Instructions: Reference

opcode			
imm[20 10:1 11 19:12]	rd	1101111	jal



Jump Instructions II: `jalr`

- Updating the Program Counter (PC)
- B-Type Instructions
- Jump Instructions
- Jump Instruction II: `jalr`
- U-Type Instructions:
`lui`, `auipc`



jalr

- **J**ump **a**nd **l**ink **r**egister

```
jalr rd rs1 imm
```

$$R[rd] = PC + 4$$
$$PC = R[rs1] + imm$$

- Means:

- Write $PC + 4$ to register rd
- Unconditional jump: go to statement at address $R[rs1] + imm$

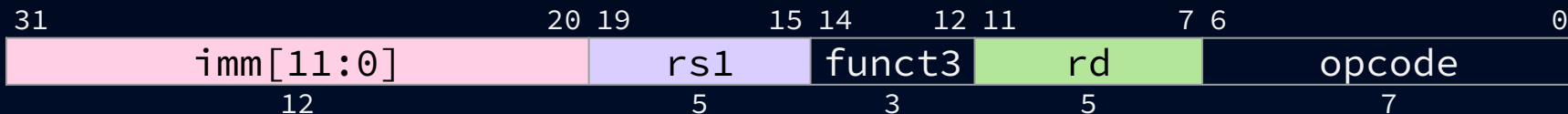
- Used for:

- Returning from procedures
 - Pseudo **jr rs1**
 $\rightarrow \text{jalr } x0 \text{ } rs1 \text{ } 0$
 - Pseudo **ret**
 $\rightarrow \text{jalr } x0 \text{ } ra \text{ } 0$
- Extra-long jumps with absolute addressing



RV32I I-Type: jalr

jalr rd rs1 imm



Imm[11:0] "offset"

Add to base register to
form target PC address

"Base" Register
Base address of
jump

"Destination" Register
gets "return address."
 $R[rd] = PC + 4$

$$PC = R[rs1] + imm$$

$$PC = (\text{Base Register}) + (\text{Immediate Offset})$$

Conditionally Branching with Jumps

- B-Type (branches) cover most cases of if-else/loops:
 - $\pm 2^{10}$ 32-bit instructions from current instruction
 - Control flow usually about <50 lines of code
- Can branch further with J-Type (j, jal):
 - $\pm 2^{18}$ 32-bit instructions from current instruction
 - Say, to libraries
- Can branch even further with **jalr** (I-Type)
 - **Absolute addressing**, so use sparingly
 - auipc + jalr (up next)



U-Type Instructions: `lui, auipc`

- Updating the Program Counter (PC)
- B-Type Instructions
- Jump Instructions
- Jump Instruction II: `jalr`
- U-Type Instructions:
`lui, auipc`



lui, auipc Both Load Upper Immediates

- **L**oad **u**pper **i**mmEDIATE

lui rd immu

$\text{imm} = \text{immu} \ll 12$

$R[\text{rd}] = \text{imm}$

- Means:

- Write an upper immediate (20-bit immediate value in the upper 20-bits) to register rd.
- Clear the lower 12 bits of register rd.

- **A**dd **u**pper **i**mmEDIATE to **P**C

auipc rd immu

$\text{imm} = \text{immu} \ll 12$

$R[\text{rd}] = \text{PC} + \text{imm}$

- Means:

- Add an upper immediate to the PC.

(how to use? More later)



U-Type Instruction Format

- Upper immediate instructions
 - `lui, auipc`

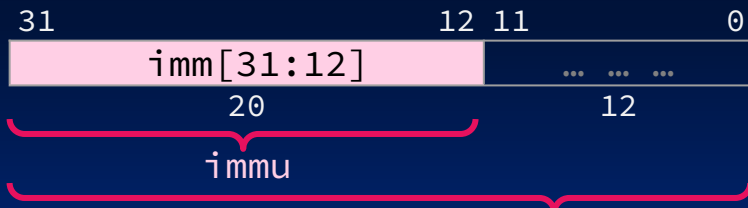
opname rd immu



`immu = imm[31:12]`

- `immu` represents the upper 20 bits of a 32-bit immediate.
- `imm = immu << 12`

"Destination" Register gets result of operation.





lui + addi creates Long Immediates

- lui together with addi can create any 32-bit value in a register.
 - lui: set upper 20 bits (and zero out lower 12 bits)
 - addi: "set" lower 12 bits (by adding **sign-extended** 12-bit immediate)
- The **li pseudoinstruction** (load immediate) resolves to lui + addi as needed.

li x10 0x87654321



lui x10 0x87654
addi x10 x10 0x321

x10

~~0x87654000~~

0x87654321

li x10 0xB0BACAFE



⚠ hm...

lui + addi Edge Case: addi Extends Sign

- How to translate lui x10 0xB0BACAFE?

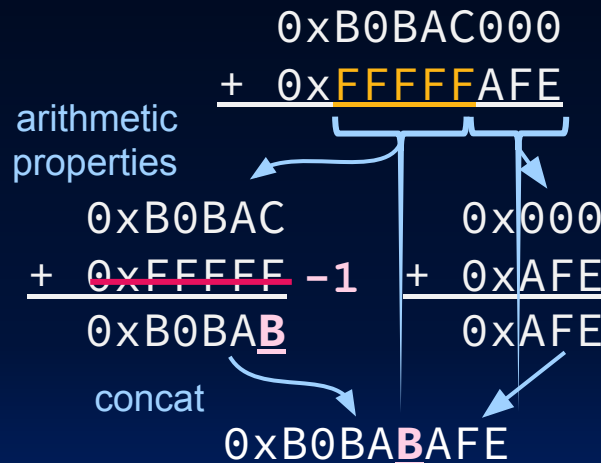
- Unfortunately:

lui x10 0xB0BAC
 addi x10 x10 0xAFF

x10 0xB0BAC000

 0xB0BABAFF

- addi **sign-extends** the 12-bit immediate.
- If **sign bit** set (12-bit immediate is negative), then **subtracts 1** from the upper 20 bits!



lui + addi Edge Case: addi Extends Sign

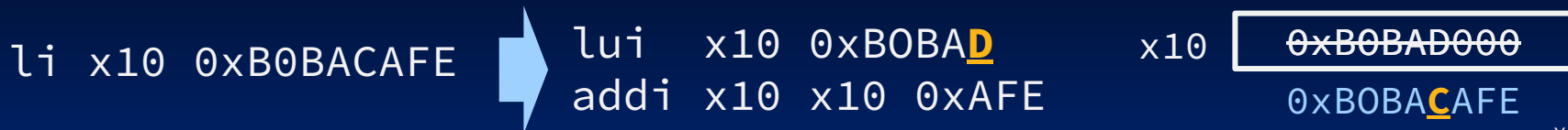
- How to translate `li x10 0xB0BACAFE`?

- Unfortunately:

<code>lui x10 0xB0BAC</code>	<code>x10</code>	<code>0xB0BAC000</code>
<code>addi x10 x10 0xAFF</code>		<code>0xB0BAC0FF</code>

- `addi` **sign-extends** the 12-bit immediate.
 - If **sign bit** set (12-bit immediate is negative), then **subtracts 1** from the upper 20 bits!

- Solution: If 12-bit immediate **is negative**, **proactively add 1** to upper 20-bit load.





U-Type Instructions: Reference

		opcode	
imm[31:12]	rd	0010111	auipc
imm[31:12]	rd	0110111	lui

"And in Conclusion..."

- We've covered (almost) the entire RV32 ISA!
- Practice assembling and disassembling!

imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
imm[20:10:11:19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12:10:5]	rs2	rs1	000	imm[4:1:11]	1100011	BEQ
imm[12:10:5]	rs2	rs1	001	imm[4:1:11]	1100011	BNE
imm[12:10:5]	rs2	rs1	100	imm[4:1:11]	1100011	BLT
imm[12:10:5]	rs2	rs1	101	imm[4:1:11]	1100011	BGE
imm[12:10:5]	rs2	rs1	110	imm[4:1:11]	1100011	BLTU
imm[12:10:5]	rs2	rs1	111	imm[4:1:11]	1100011	BGEU
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI

0000000		shamt	rs1	001	rd	0010011	SLLI
0000000		shamt	rs1	101	rd	0010011	SRLI
0100000		shamt	rs1	101	rd	0010011	SRAI
0000000		rs2	rs1	000	rd	0110011	ADD
0100000		rs2	rs1	000	rd	0110011	SUB
0000000		rs2	rs1	001	rd	0110011	SLL
0000000		rs2	rs1	010	rd	0110011	SLT
0000000		rs2	rs1	011	rd	0110011	SLTU
0000000		rs2	rs1	100	rd	0110011	XOR
0000000		rs2	rs1	101	rd	0110011	SRL
0100000		rs2	rs1	101	rd	0110011	SRA
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND
0000	pred	succ	00000	000	00000	0001111	FENCE
0000	0000	0000	00000	001	00000	0001111	FENCE.I
0000000000000			00000	000	00000	1110011	ECALL
0000000000001			00000	000	00000	1110011	EBREAK
csr			rs1	001	rd	1110011	CSRWR
csr			rs1	010	rd	1110011	CSRWS
csr			rs1	011	rd	1110011	CSRRC
csr			zimm	101	rd	1110011	CSRRIWI
csr			zimm	110	rd	1110011	CSRRIWI
csr			zimm	111	rd	1110011	CSRRCI

Not in CS61C

<https://riscv.org/technical/specifications/>

14-RISC-V Instruction Formats II (38)